AGENTICSYS V2 · DATA PLANE

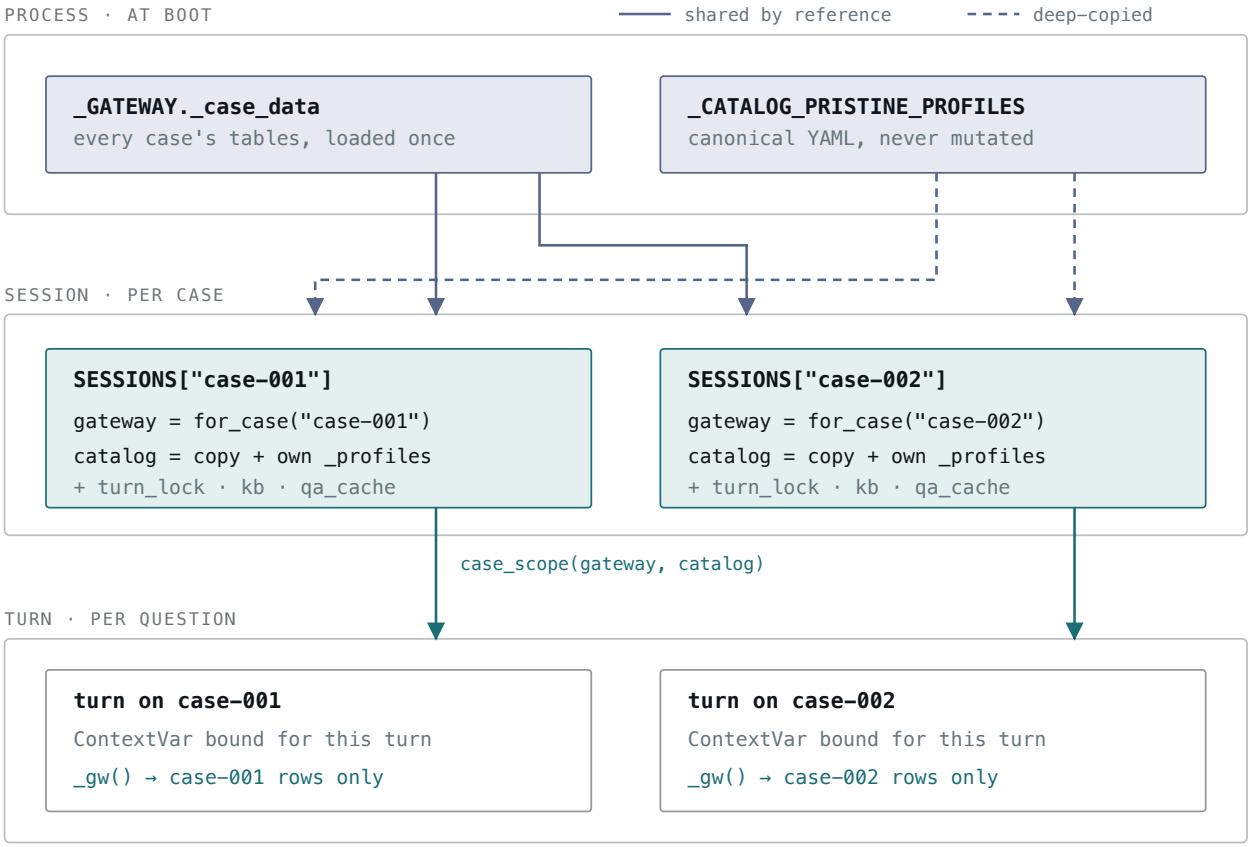
How multi-case concurrency is handled

Two reviewers, two cases, one server. What each case owns, what they share, and the one silent failure this design exists to make impossible.

Turns have always run concurrently — each one gets its own thread. What used to be unsafe was the *data plane* underneath them: one gateway carrying one `_current_case`, and one catalog whose profiles were rewritten per case. Whichever turn re-scoped last won, and the other turn kept querying under the wrong case without raising anything.

01 What exists at each level

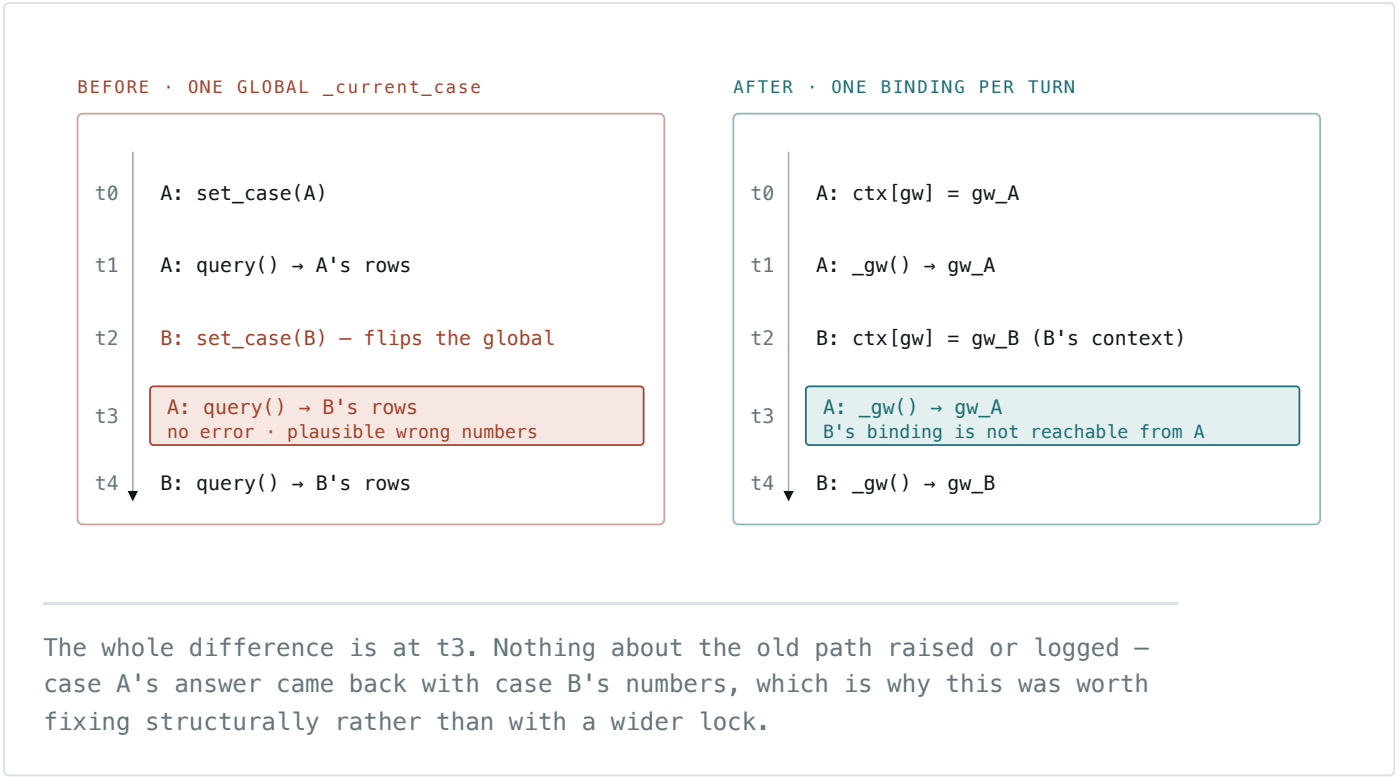
Three lifetimes, nested. The corpus is loaded once for the whole process; a gateway *view* and a catalog *copy* are built once per case; the binding that tools actually read is established once per turn and torn down when the turn ends.



Solid slate edges carry a shared reference — the row dicts are never duplicated, so N open cases cost one corpus. Dashed slate edges are copies: the catalog must be one, because the per-case sync writes this case's aliases and observed value vocabularies into `_profiles`. Petrol marks what a single case owns.

02 The failure this removes

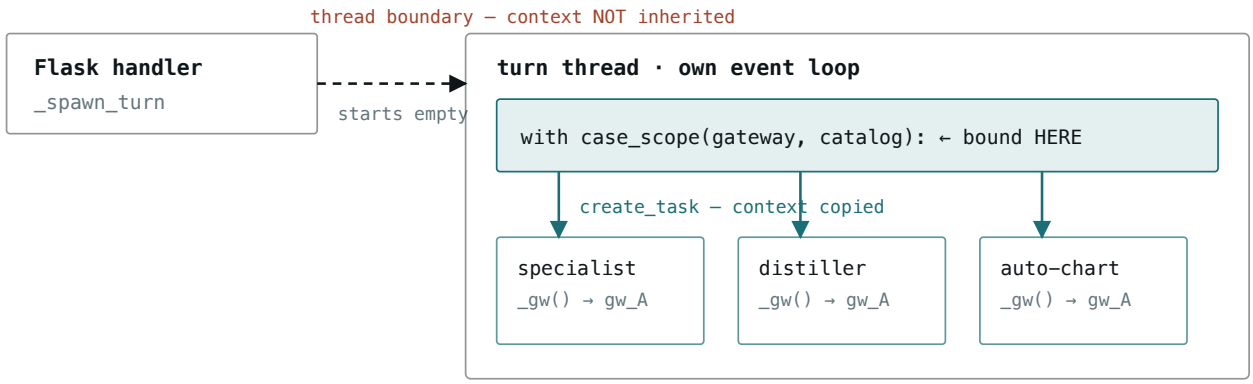
The old design re-pointed the process globals at the start of every turn. That is correct for turns taken one after another, and wrong the moment two cases overlap — because the lock that serializes turns is held *per case*, so it never stood between case A and case B at all.



The whole difference is at t3. Nothing about the old path raised or logged — case A's answer came back with case B's numbers, which is why this was worth fixing structurally rather than with a wider lock.

03 Why the binding is a ContextVar

The carrier had to match how the runtime actually forks. A turn is spawned as a thread, and inside it the orchestrator fans out to specialists, distillers and auto-chart jobs as asyncio tasks. Those two boundaries behave in opposite ways — which is exactly why the scope is entered where it is.



Because a fresh thread starts with an empty context, binding from the spawning side would never reach the turn — so the scope is entered in `_run_turn_streamed`, which already runs in the turn's own thread. Every task created after that point inherits it, which is what lets 24 helpers stay unchanged: no gateway parameter is threaded anywhere.

04 Shared versus owned

SHARED ACROSS CASES	OWNED BY ONE CASE
_case_data — the row corpus, read-only	gateway view — its own _current_case
LLM clients + firewall semaphores	catalog copy — its own _profiles
node-trace store, Amem handle	logger, specialist_kb, qa_cache
schema / search caches — keyed by case_id	turn_lock, cancel_in_flight, event buffer

ONE ID, ONE SPELLING

A case id enters through three doors — a directory name, a CSV column, a URL segment — and none of them promises a clean string. Since the id builds paths (`reports/<case_id>/charts/`, `logs/case-<case_id>-…`), one case arriving under two spellings would fork into two report directories and two log files. `normalize_case_id` runs at every ingress, and `set_case` is the single

assignment point that applies it — so a padded id still resolves to the loaded case rather than silently querying one that doesn't exist.

---

05 What this does not buy

> CONCURRENCY IS BOUNDED BY LLM CAPACITY, NOT CORRECTNESS
>
> The firewall semaphores are process-wide — 12 specialist slots, 2 orchestrator slots. Three cases running at once still queue at the planning phase. Correct in parallel, not linearly faster.

> SAME-CASE TURNS STILL SERIALIZE
>
> `turn_lock` is per case and unchanged: a second question on a case waits for the first, up to `queued_turn_max_wait_s`. That is deliberate — the session's KB and episodic ordering assume one turn at a time.

> VIEWS SHARE ROW DICTS READ-ONLY
>
> Every current tool filters and projects rather than mutating. A future tool that edited a row in place would corrupt every other session holding that case.

---

06 Where it lives

datalayer/gateway.py — `for_case()`, `normalize_case_id()`
tools/data_tools.py — `case_scope()`, `_gw()`, `_cat()`
server.py — `_get_or_create_session()` builds the pair, `_case_scope()` binds it
tests — `test_gateway_for_case.py`, `test_case_scope.py`, `test_server.py`